
Práctica 12: iluminación y reflejos

Fecha de entrega: 28 de abril de 2026, 13:50

Objetivo: Extensión de un trazador de rayos con iluminación y reflejos

1. Instrucciones

En esta práctica partiremos del resultado de la práctica anterior en la que se construyó el esqueleto básico de un trazador de rayos e iremos aplicando extensiones encaminadas a añadir soporte para luces direccionales y puntuales (estas últimas generando sombras) y reflejos.

Las extensiones se van describiendo poco a poco más abajo. Aunque se pasará por distintas “versiones” del trazador, sólo será necesario incluir la versión final del código. La entrega debe incluir esos ficheros de tal forma que baste compilar todos los `.cpp` desde el directorio raíz de código (en Linux debería bastar con algo como `g++ *.cpp`).

En la entrega será necesario, además, incluir el resultado de renderizado tras la implementación de algunos de los apartados. En todos los casos la captura será en resolución FullHD (1920×1080) y será una imagen en formato PNG.

2. Punto de partida

El punto de partida es una escena parecida a la final de la práctica anterior. Modifica el `main` para tener:

- La cámara colocada en $(0, 0, 3)$ mirando hacia $(0, 0, 0)$ y con una apertura de foco de 60 grados.
- Tres esferas de radio 1, colocadas en $(-2, 0, -2)$, $(0, 0, -2)$ y $(2, 0, -2)$. Los colores de cada una serán roja, amarilla y azul, respectivamente.
- Una esfera verde haciendo las veces de “suelo”, colocada en $(0, -100, -2)$ y de radio 99.

Además, eliminamos la simulación del “skybox” que hacía el *renderer*, de forma que si el rayo no se colisiona con nada el color resultante será el negro.

El resultado esperado de la escena renderizado con la práctica anterior se muestra en la figura 1.

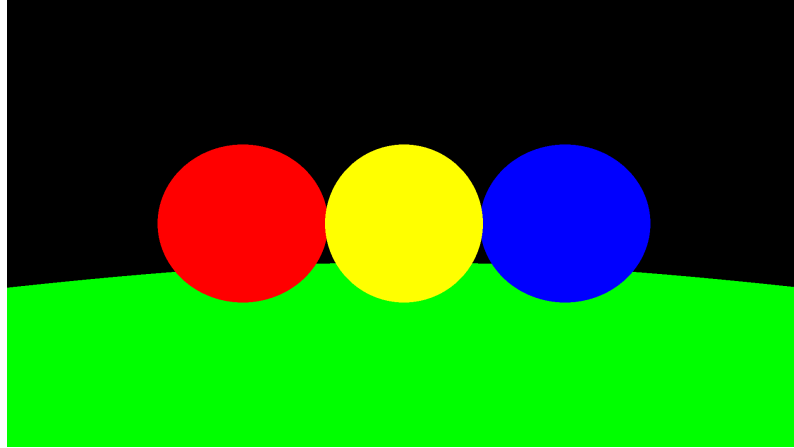


Figura 1: Trazador de rayos de partida

3. Soporte para luces

El primer paso es añadir soporte para luces en el trazador. Para eso necesitamos:

- Una clase `Light` que represente una luz. Tendrá un método encargado “iluminar” un punto en base a sus propiedades.
- Una clase `World` que tenga la escena (`Shape`) a renderizar y el conjunto de luces.
- Modificar la clase `Renderer` para recibir un objeto de la clase anterior en lugar de únicamente la geometría.
- Extender el método de renderizado para que el color de cada punto se calcule teniendo en cuenta las luces que lo iluminan y las propiedades del material.
- En la implementación de las luces necesitarás que `ShapeIntersection` tenga, además de la información que ya tiene, la normal de la superficie con la que colisiona el rayo.

El método de la clase `Light` encargado de iluminar (puede llamarse por ejemplo `shade`) recibirá el rayo que provoca el cálculo (que viene desde la cámara) y la información de la colisión con el objeto (`ShapeIntersection` en la práctica anterior) y devuelve el color resultante.

El color de un rayo que choca con una superficie se calcula:

```
Color Renderer::shade(Ray r, ShapeIntersection hit) {

    Color ret;

    // Ambiente
    ret += Color(0.1, 0.1, 0.1); // Luz de ambiente, cableada
```

```
// luces
para cada luz l en el mundo:
    color += l->shade(r, hit);

return ret;
}
```

4. Luz direccional

Crea una clase `DirectionalLight` que implemente luces direccionales. En el constructor recibirá tanto la dirección como el color base de la luz. Puedes hacer la implementación en dos pasos, primero añadiendo únicamente la componente difusa y posteriormente la especular. Para el cálculo puede servirte las transparencias de iluminación dadas en teoría o los shaders implementados en la práctica 4. Recuerda que el resultado del cálculo difuso es multiplicado por el color base del material pero el especular no.

Prueba la luz añadiendo una de color blanco en la posición (1, 1, 0). Genera una imagen del resultado para incorporarla a la entrega con el nombre `01_LuzDireccional.png`.

5. Luz puntual

Crea una nueva clase `PointLight` para implementar luces puntuales que reciban en el constructor la posición y el color.

Para probarlo, quita la luz direccional y añade una luz puntual en la posición (2, 2, 0).

6. Sombras

Las luces puntuales pueden generar sombras. Extiende la implementación del trazador para hacerlo. Necesitarás:

- Extender el interfaz de la luz con dos métodos, uno que permita averiguar si la luz genera sombras o no y otro que, en caso de hacerlo, calcule la dirección del vector que une un punto cualquiera de una superficie con la luz.
- Modificar el cálculo de iluminación de cada punto de forma que para cada luz que proyecte sombras, si hay algún objeto entre el punto y la luz, su color no se tenga en cuenta.

```
Color Renderer::shade(Ray r, ShapeIntersection hit) {

    Color ret;

    // Ambiente
    ret += Color(0.1, 0.1, 0.1); // Luz de ambiente, cableada

    // luces
    para cada luz l en el mundo:
        si l proyecta sombras:
            shadowRay = ray(hit.pos, l.shadowDir(hit.pos))
            si shadowRay colisiona con geometria:
```

```

        saltar a la siguiente luz
        color += l->shade(r, hit);

    return ret;
}

```

Prueba las sombras con la escena anterior. Puedes además añadir una segunda luz puntual en (-2, 2, -3).

Genera una imagen con el resultado con el nombre 02_Sombras.png e incorpórala a la entrega.

7. Reflejo simple

Añade la posibilidad de que un material refleje parcialmente la luz. Para eso, incorpora un factor de reflexión en el material de forma que cuando se calcula el color del punto, si éste es mayor que 0 se calcule el rayo reflejado en el punto y se obtenga su color. Ese color será entonces multiplicado por el factor y sumado al color final.

```

Color Renderer::shade(Ray r, ShapeIntersection hit) {

    Color ret;

    // Ambiente
    ret += Color(0.1, 0.1, 0.1); // Luz de ambiente, cableada

    // luces
    para cada luz l en el mundo:
        ...

    // reflejo
    si hit.material.factorReflejo > 0:
        Ray secundario = ray(hit.pos, reflect(hit.pos, hit.normal));
        ret += hit.material.factorReflejo * ray_color(secundario);

    return ret;
}

```

Prueba la nueva característica incorporando al suelo (esfera verde) un factor de reflejo de 0.5.

Genera una imagen con el resultado de nombre 03_ReflejoSimple.png e incorpórala a la entrega con

8. Reflejo múltiple

Cambia la escena para que los materiales del resto de esferas tengan también un factor de reflejo de 0.5.

Para evitar una posible recursión infinita, limita el número de llamadas a, por ejemplo, 10.

Genera una imagen con el resultado (04_Final.png) e incorpórala a la entrega.

9. Aplica tu imaginación

Aprovecha las nuevas características y crea alguna escena nueva y renderizala. Incorporalas a la entrega en un directorio `EscenasAdicionales` incluyendo tanto el PNG con el resultado final como el `main.cpp` que construye la escena.

10. Entrega de la práctica

La práctica debe entregarse utilizando el mecanismo de entrega del campus virtual, no más tarde de la fecha y hora indicada en la cabecera de la práctica.

Solo uno de los dos miembros del grupo debe hacer la entrega, subiendo al campus un fichero llamado `GrupoNN.zip` donde NN representa el número de grupo con dos dígitos.

El fichero debe tener exclusivamente:

- Un directorio `Src` con el código fuente completo del trazador de rayos. El `main` debe construir una escena similar a la del apartado 8. El código debe compilar tal y como se ha descrito en la sección 1.
- Una imagen con el resultado de dibujar la escena con el trazador de rayos implementado.
- Un fichero `alumnos.txt` donde se indicará el nombre de los componentes del grupo.